UNIVERSITÉ DE ROUEN NORMANDIE
FACULTÉ DES SCIENCES ET TECHNIQUES

ANNÉE UNIVERSITAIRE 2025–2026

Architecture Decision Records

Système de Gestion de Flotte

Semaine 1

11 mars 2026

Contents

1 ADR-001 — Services indépendants **3**

2 ADR-002 — Protocole rapide pour le GPS **3**

3 ADR-003 — Alertes traitées sans bloquer le suivi **3**

4 ADR-004 — Une base adaptée à chaque type de donnée **3**

5 ADR-005 — Connexion et droits centralisés **3**

6 ADR-006 - Keycloak **4**

7 Configuration Keycloak — Semaine 1 **5**

7.1 1. Déploiement du serveur Keycloak en local 5

7.2 2. Création du Realm 5

7.3 3. Mise en place du contrôle d'accès (RBAC) 5

7.4 4. Configuration des Clients 5

7.5 5. Création d'un utilisateur de test 6

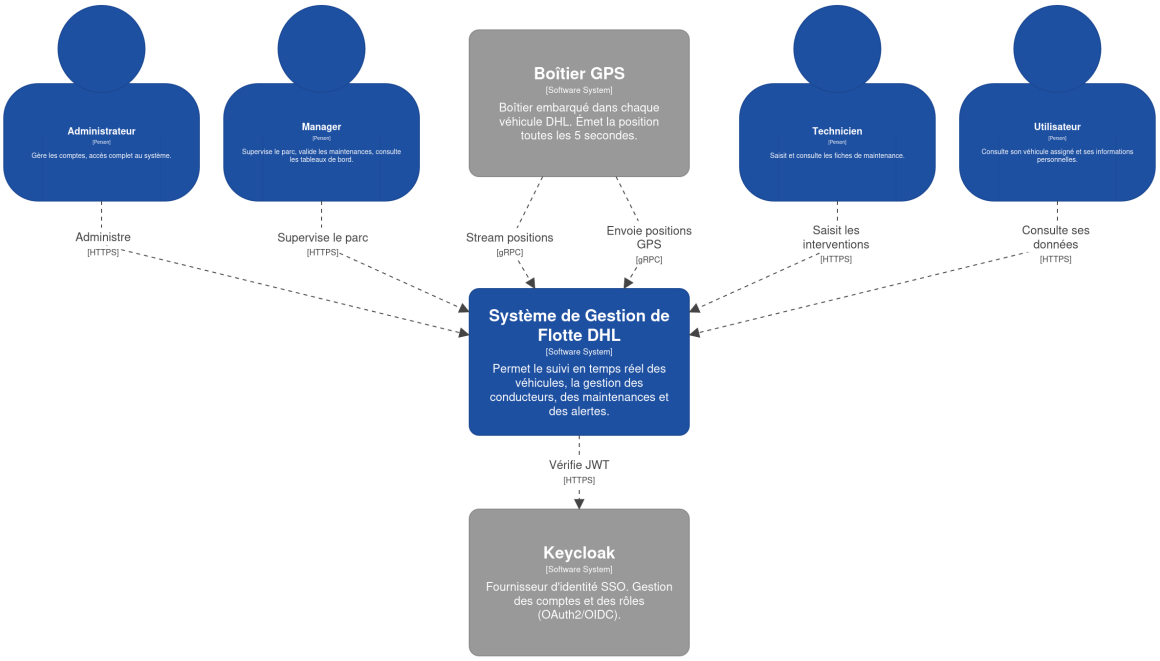
7.6 6. Exportation de la configuration (Livrable) 7

Architecture Decision Records

Système de Gestion de Flotte — DHL Express

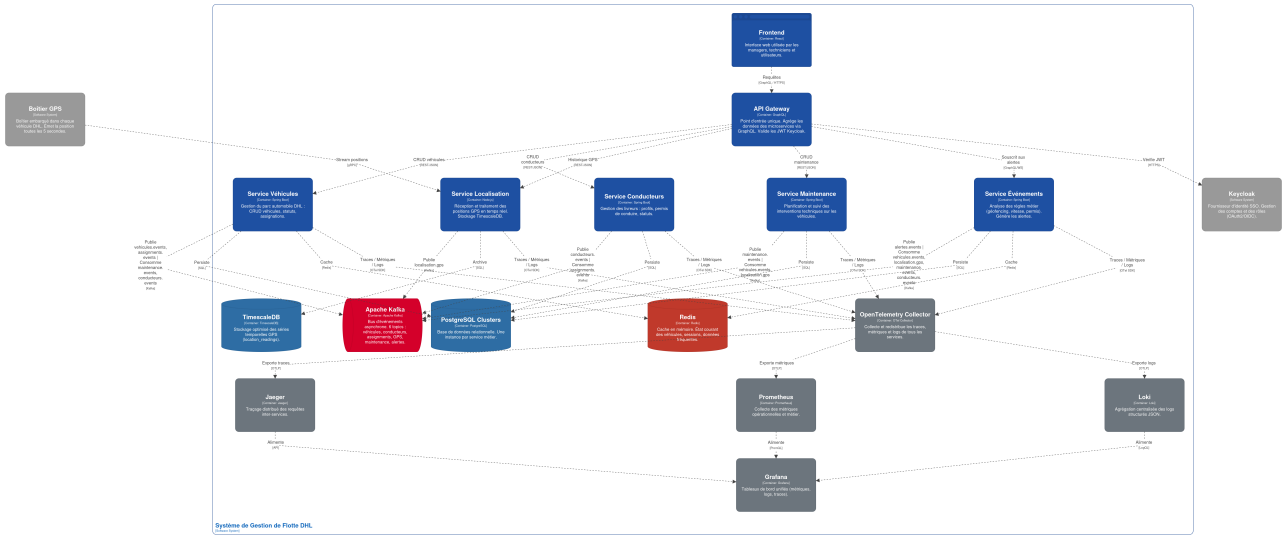
Ce document recense les décisions d'architecture prises en semaine 1 pour le système de **suivi en temps réel de la flotte de véhicules DHL Express**. Le système couvre la gestion des livreurs, l'affectation des véhicules aux tournées, la maintenance du parc et les alertes automatiques. Les utilisateurs sont les **administrateurs**, **managers**, **techniciens** et **utilisateurs**.

C1 — Vue système



C1 — Contexte système : Gestion de Flotte DHL
Vue d'ensemble des acteurs et systèmes externes interagissant avec le système.
samedi 14 mars 2026 à 10:16 heure normale d'Europe centrale

C2 — Architecture des conteneurs



C2 — Conteneurs : Gestion de Flotte DHL
Décomposition interne du système en microservices, bases de données et infrastructure.
samedi 14 mars 2026 à 10:16 heure normale d'Europe centrale

ADR-001 — Services indépendants

Le système doit faire trois choses à la fois : suivre les véhicules en direct, gérer les données administratives et envoyer des alertes. Ces trois besoins sont très différents et évolueront indépendamment.

Décision : séparer le système en trois parties indépendantes (Localisation, Administratif, Alertes), toutes accessibles depuis une seule adresse. **Avantage :** si une partie tombe en panne, les autres continuent de fonctionner. **Inconvénient :** un peu plus de choses à gérer au démarrage.

ADR-002 — Protocole rapide pour le GPS

Les boîtiers GPS envoient leur position plusieurs fois par seconde. Le protocole web classique n'est pas fait pour gérer autant de données en continu.

Décision : utiliser **gRPC**, un protocole conçu pour transmettre de grandes quantités de données rapidement. Les actions classiques (consulter, modifier) restent en HTTP normal. **Avantage :** les positions s'affichent sans délai perceptible. **Inconvénient :** une petite couche de traduction est nécessaire pour que le navigateur comprenne.

ADR-003 — Alertes traitées sans bloquer le suivi

Quand un véhicule dépasse une limite de vitesse ou entre dans une zone interdite, une alerte doit être envoyée. Cette étape ne doit pas ralentir le suivi GPS.

Décision : utiliser **Apache Kafka**, une file d'attente qui stocke les alertes à envoyer. Le suivi GPS dépose l'alerte, et le service d'envoi la traite quand il est prêt. **Avantage :** les deux parties travaillent sans se bloquer mutuellement, et aucune alerte n'est perdue. **Inconvénient :** un outil de plus à installer et surveiller.

ADR-004 — Une base adaptée à chaque type de donnée

Les positions GPS s'accumulent très rapidement, les données métier (véhicules, conducteurs) sont structurées et liées entre elles, et certaines informations sont consultées si souvent qu'elles doivent répondre en un instant.

Décision : utiliser trois bases de données, chacune adaptée à son rôle.

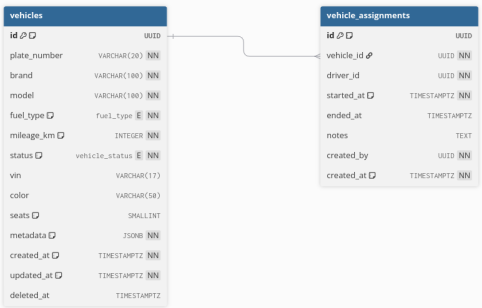
Outil	Rôle
TimescaleDB	Historique des positions GPS (conçu pour stocker efficacement des données qui arrivent en continu)
PostgreSQL	Véhicules, conducteurs, maintenances, alertes
Redis	Données consultées très souvent, stockées en mémoire pour répondre instantanément

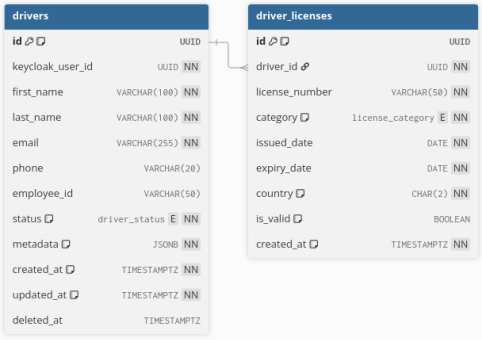
ADR-005 — Connexion et droits centralisés

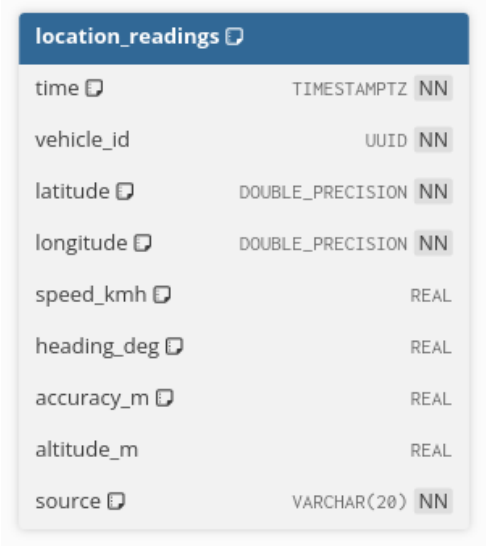
Administrateurs, managers et techniciens n'ont pas accès aux mêmes fonctionnalités. Il faut un moyen simple de gérer qui peut faire quoi, sans le répéter dans chaque partie du système.

Décision : utiliser **Keycloak**, un outil gratuit qui centralise la gestion des comptes et des droits. L'utilisateur se connecte une seule fois et peut accéder à toutes les parties du système selon son rôle. **Avantage :** les droits sont gérés en un seul endroit. **Inconvénient :** si cet outil est en panne, plus personne ne peut se connecter — il doit donc toujours être disponible.

Schémas de base de données (PostgreSQL)







ADR-006 - Keycloak

Configuration Keycloak — Semaine 1

Cette section documente les étapes de mise en place de Keycloak réalisées en semaine 1, constituant le livrable de sécurité du projet.

7.1 1. Déploiement du serveur Keycloak en local

Pour l'environnement de développement, Keycloak a été conteneurisé et lancé via Docker avec la création automatique d'un compte super-administrateur. La commande suivante démarre Keycloak en mode développement sur le port 8080 :

```
docker run -p 8080:8080
  -e KEYCLOAK_ADMIN=admin
  -e KEYCLOAK_ADMIN_PASSWORD=admin
  quay.io/keycloak/keycloak:latest start-dev
```

L'interface d'administration est ensuite accessible sur `http://localhost:8080`.

7.2 2. Création du Realm

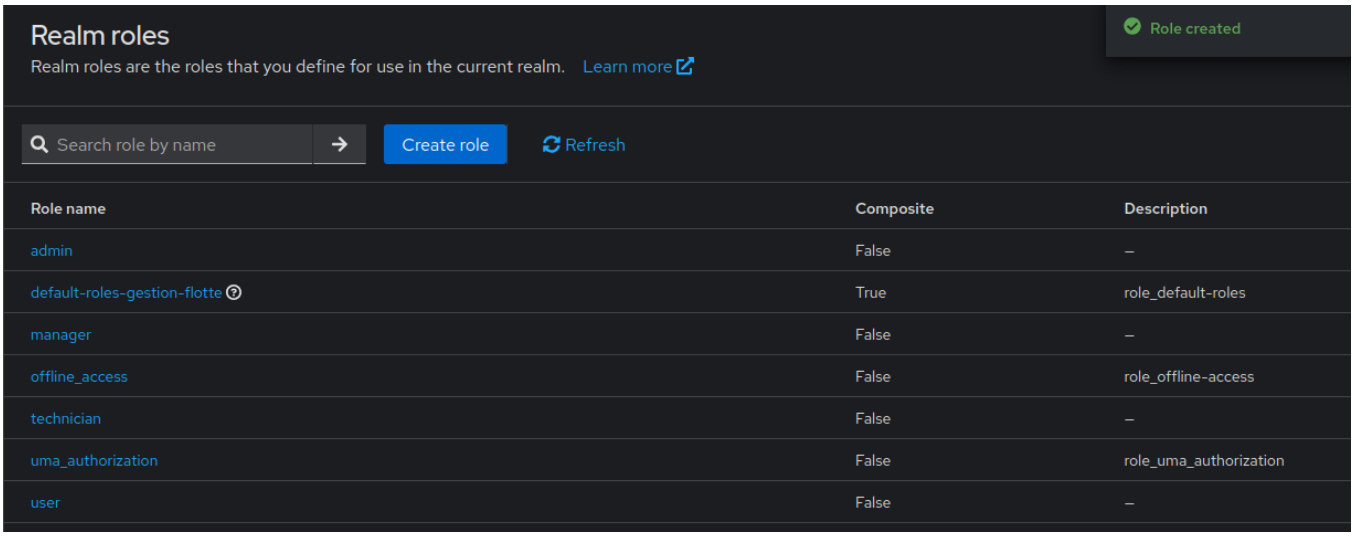
Afin d'isoler notre environnement du realm `master` (réservé à l'administration de l'outil), un nouveau Realm dédié au projet a été créé :

- **Nom du Realm :** `gestion-flotte`

7.3 3. Mise en place du contrôle d'accès (RBAC)

Les rôles métier suivants ont été définis dans la section *Realm roles* :

Rôle	Périmètre
`admin`	Accès complet à tous les services et données
`manager`	Supervision du parc, validation des maintenances
`technicien`	Lecture/écriture sur les fiches de maintenance
`user`	Consultation de ses propres données uniquement



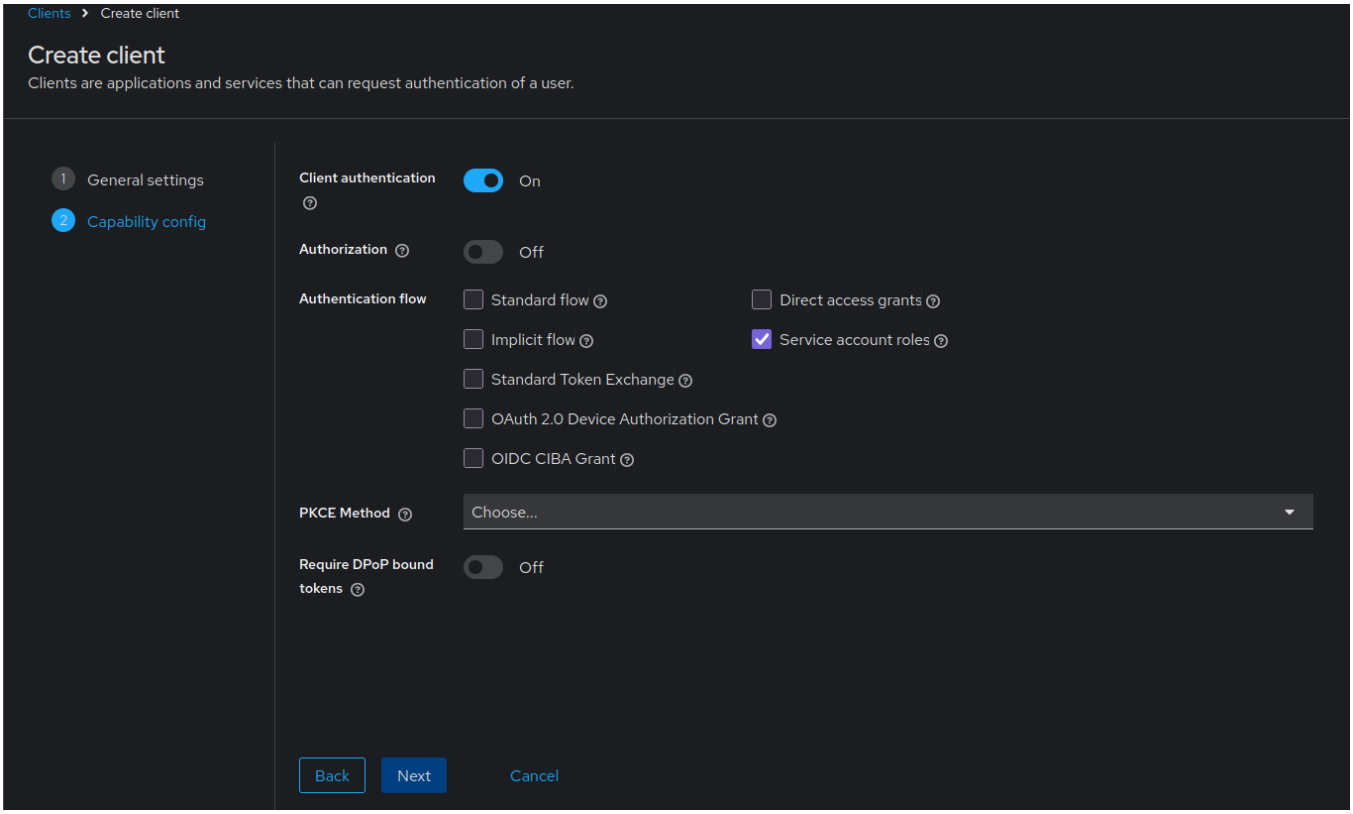
Liste des Realm roles créés dans le realm `gestion-flotte`

7.4 4. Configuration des Clients

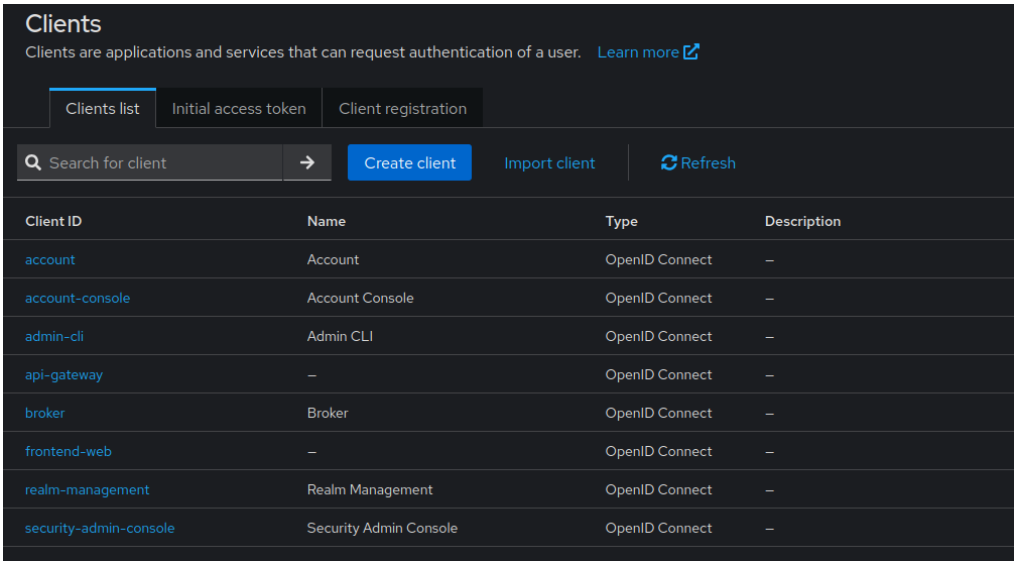
Deux clients ont été configurés pour interagir avec Keycloak :

Client 1 : `frontend-web` **(Client Public)** — Destiné à l'interface React/Vue. Configuration : Standard flow activé, redirection autorisée vers $*$ pour le développement.

Client 2 : `api-gateway` **(Client Confidentiel)** — Destiné à la communication serveur à serveur. Configuration : *Client authentication* activée, *Service accounts roles* activés, génération d'un *Client Secret* pour sécuriser les échanges GraphQL.



Liste des clients configurés dans le realm `gestion-flotte`

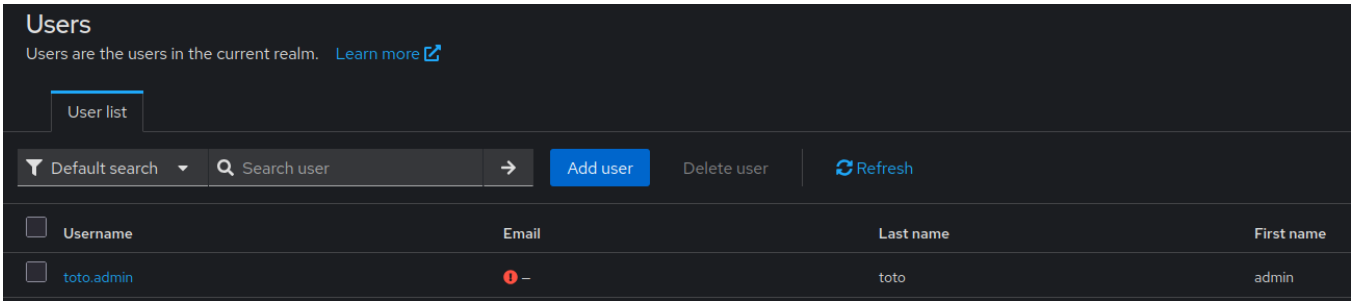


Configuration du client `api-gateway` — Client authentication ON, Service account roles activé

7.5 5. Création d'un utilisateur de test

Un utilisateur de test a été provisionné pour valider les futurs développements :

Paramètre	Valeur
Identifiant	`toto.admin`
Mot de passe	Défini manuellement, *Temporary = OFF*
Role mapping	Rôle `admin` assigné



Liste des utilisateurs du realm `gestion-flotte` — utilisateur `toto.admin` créé avec le rôle `admin`

7.6 6. Exportation de la configuration (Livrable)

Pour garantir la reproductibilité de l'environnement (*Infrastructure as Code*), l'intégralité de la configuration a été exportée via l'interface Keycloak (Include clients + Include groups and roles). Le fichier généré `realm-export.json` est versionné dans le dépôt Git du projet sous le chemin `infrastructure/keycloak/realm-export.json`, ce qui permet de reconstruire l'environnement Keycloak complet automatiquement lors du déploiement en semaine 2.